

Лабораторная работа №3

Моделирование графов атак

Бабенко Артём Сергеевич

Содержание

1	Цель работы	4
2	Задание	5
3	Теоретическое введение	6
4	Выполнение лабораторной работы	7
4.1	Подготовка каталога	7
4.2	Ядро моделирования графа атак	8
4.3	Однократный запуск эксперимента	9
4.4	Визуализация и анализ графа	10
4.5	Исследование масштабируемости	12
4.6	Параметрическое исследование	14
4.7	Преобразование кода в литературный стиль	15
4.8	Изменение кода	18
4.9	Реализация модуля AttackGraph	21
4.10	Запуск эксперимента	22
4.11	Анализ и визуализация	22
4.12	Исследование масштабируемости	23
4.13	Параметрическое исследование	24
5	Контрольные вопросы	25
6	Выводы	27
	Список литературы	28

Список иллюстраций

4.1	Содержимое каталога	7
4.2	Содержимое каталога	7
4.3	Установка пакетов	8
4.4	Скрипт <code>src/attack_graph.jl</code>	8
4.5	Скрипт <code>ag_run_experiment.jl</code>	9
4.6	Результат выполнения <code>ag_run_experiment.jl</code>	9
4.7	Скрипт <code>ag_analyze.jl</code>	10
4.8	Результат выполнения <code>ag_analyze.jl</code>	10
4.9	Визуализация графа	11
4.10	Визуализация графа	11
4.11	Скрипт <code>ag_convergence.jl</code>	12
4.12	Рост времени и числа путей	13
4.13	Рост времени и числа путей	13
4.14	Скрипт <code>parameter_sweep.jl</code>	14
4.15	Зависимость от плотности рёбер	14
4.16	Зависимость от плотности рёбер	15
4.17	Преобразование кода	16
4.18	Генерация производных форматов	17
4.19	Выполнение Jupyter Notebook	17
4.20	Выполнение Jupyter Notebook	18
4.21	Интеграция Quarto	18
4.22	Изменение кода	19
4.23	Выполнение Jupyter Notebook	19
4.24	Выполнение Jupyter Notebook	20
4.25	Выполнение Jupyter Notebook	20

1 Цель работы

- Освоить методы построения и анализа графов атак для оценки уязвимостей сетевой инфраструктуры.
- На примере моделирования атак на корпоративную сеть изучить:
 - представление сетевой топологии и уязвимостей в виде ориентированного графа;
 - алгоритмы поиска всех возможных путей атаки от начальных точек до целевых активов;
 - расчёт метрик центральности для определения критических узлов;
 - визуализацию графа с цветовой индикацией уровня риска;
 - оценку вероятности успешной атаки с учётом сложности эксплуатации уязвимостей.

2 Задание

- Построить граф атак для заданной топологии сети.
- Реализовать алгоритм поиска всех путей от заданного источника к цели.
- Рассчитать метрики центральности для всех узлов и выявить наиболее критичные.
- Визуализировать граф, раскрашивая узлы в зависимости от степени риска.
- Присвоить каждому ребру вес (вероятность успешной атаки) и вычислить наиболее вероятный путь атаки.

3 Теоретическое введение

Граф атак (Attack Graph) — это ориентированный граф, в котором вершины представляют состояния системы (узлы сети, привилегии, факты), а рёбра — действия, которые может выполнить атакующий для перехода из одного состояния в другое. В упрощённой модели граф атак можно представить как ориентированный граф, где:

- вершины — сетевые узлы (хост, сервер, рабочая станция);
- направленные рёбра — возможность атаки с одного узла на другой (через эксплойт уязвимости, доверительные отношения, сетевой доступ).

Для каждого узла можно задать уровень защищённости, список уязвимостей и тип узла.

Задача анализа графа атак сводится к нахождению всех путей от источника (злоумышленник) к цели (критический актив). Для этого применяются алгоритмы поиска в глубину (DFS), в ширину (BFS) и алгоритм Дейкстры для поиска наименее сложного пути.

Для выявления критичных узлов используются метрики центральности: степень центральности (in-degree, out-degree), центральность по посредничеству, по близости и PageRank.

Если каждому ребру присвоить вероятность успешной эксплуатации, то вероятность успешной атаки по пути вычисляется как произведение вероятностей рёбер.

4 Выполнение лабораторной работы

4.1 Подготовка каталога

Создадим рабочий каталог для кода.

```
julia> initialize_project("project"; authors="Bams Ims", git=false)
Activating new project at 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\labs\lab02\project'
Resolving package versions...
Installed ScopedValues - v1.6.0
Installed JLD2 - v0.6.4
Updating 'C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\labs\lab02\project\Project.toml'
[634d3b9d] + DrWatson v2.19.1
[0b6fb165] + ChunkCodeCore v1.0.1
[4c0bbe4] + ChunkCodeLibZlib v1.0.0
[55437552] + ChunkCodeLibZstd v1.0.0
[634d3b9d] + DrWatson v2.19.1
[5789e2e9] + FileIO v1.18.0
[976d061b] + HashArrayMappedTries v0.2.0
[033835bb] + JLD2 v0.6.4
[692b3bcd] + JLIWrappers v1.7.1
[1914dd2f] + MacroTools v0.5.16
[bac558e1] + OrderedCollections v1.8.1
x [aea7be01] + PrecompileTools v1.2.1
[21216c6a] + Preferences v1.5.2
[ae029012] + Requires v1.3.1
[7e506255] + ScopedValues v1.6.0
[6c6a2e73] + Scratch v1.3.0
[3a884ed6] + UnPack v1.0.2
[3161d3a3] + Zstd_jll v1.5.7+1
[0dad04e5] + ArgTools v1.1.1
```

Рисунок 4.1: Содержимое каталога

Содержимое каталога.

```
PS C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\labs\lab02\project> ls

Каталог: C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\labs\lab02\project

Mode                LastWriteTime         Length Name
----                -
d-----          30.03.2026         14:13      .github
d-----          30.03.2026         14:13      data
d-----          30.03.2026         14:13     notebooks
d-----          30.03.2026         14:13      papers
d-----          30.03.2026         14:13      plots
d-----          30.03.2026         14:13     scripts
d-----          30.03.2026         14:13      src
d-----          30.03.2026         14:13      test
d-----          30.03.2026         14:13     _research
-a-----          30.03.2026         4722 .gitattributes
-a-----          30.03.2026         6247 .gitignore
-a-----          30.03.2026         6766 Manifest.toml
-a-----          30.03.2026          150 Project.toml
-a-----          30.03.2026         1013 README.md
```

Рисунок 4.2: Содержимое каталога

Установим необходимые пакеты.

```

using Pkg
Pkg.activate(".")
# Список пакетов для установки
packages = ["DrWatson", "Graphs", "GraphRecipes", "Plots", "JLD2",
"CSV", "DataFrames", "StatsBase", "Random", "LinearAlgebra",
"Literate", "IJulia"]

println("=", ^60)
println("Установка пакетов для моделирования графов атак")
println("=", ^60)

# Устанавливаем пакеты
for pkg in packages
    println("Установка: $pkg...")
    Pkg.add(pkg)
end

```

Рисунок 4.3: Установка пакетов

4.2 Ядро моделирования графа атак

Создадим ядро моделирования графа атак, которое содержит все функции для построения ориентированного графа атак, поиска путей, расчёта метрик центральности, присвоения весов и нахождения наиболее вероятного пути.

```

attack_graph.jl

using Graphs, LinearAlgebra

function build_attack_graph(n, edge_prob, vulnerabilities, trust_relations)
    g = SimpleDiGraph{n}
    # случайные рёбра
    for i = 1:n, j = 1:n
        if i != j && rand() < edge_prob
            add_edge!(g, i, j)
        end
    end
    # доверительные отношения
    for (u, v) in trust_relations
        add_edge!(g, u, v)
    end
    return g
end

function find_all_paths(g, source, target)
    paths = []
    function dfs(current, path)
        if current == target
            push!(paths, copy(path))
            return
        end
        for v in neighbors_out(g, current)
            push!(path, v)
            dfs(v, path)
            pop!(path)
        end
    end
    push!(path, source)
    dfs(source, path)
    return paths
end

```

Рисунок 4.4: Скрипт src/attack_graph.jl

4.3 Однократный запуск эксперимента

Напишем скрипт `scripts/ag_run_experiment.jl`, который выполняет построение графа атак для заданных параметров, находит все пути, вычисляет метрики центральности, определяет наиболее вероятный путь и сохраняет результаты.

```
> julia> ag_run_experiment.jl
using DrWatson
@quickactivate "project"
using Graphs, JLD2, Random
include(scrdir("attack_graph.jl"))
params = Dict{
    :n => 20,
    :edge_prob => 0.2,
    :source => 1,
    :target => 20,
    :cvss_scores => Dict{Tuple{Int,Int}}{Float64}((1,3) => 0.9, (2,5) => 0.7, (3,8) => 0.8, (5,20) => 0.95),
    :trust_relations => [(4,6), (6,10), (10,15)],
}

filename = datadir("attack_graph", savename(params, "jld2"))
mkpath(datadir("attack_graph"))

if isfile(filename)
    @load filename data
    println("Данные загружены из $filename")
else
    g = build_attack_graph(
        params[:n],
        params[:edge_prob],
        params[:cvss_scores],
        params[:trust_relations],
    )
```

Рисунок 4.5: Скрипт `ag_run_experiment.jl`

```
julia> include("scripts/ag_run_experiment.jl")
Результаты сохранены в C:\Users\Admin\work\study\2026-2027\...
e_prob=0.2_n=20_source=1_target=20.jld2
Количество путей атаки: 5316
Наиболее вероятный путь: [1, 16, 19, 20]
Вероятность успеха: 0.12500000000000003
```

Рисунок 4.6: Результат выполнения `ag_run_experiment.jl`

4.4 Визуализация и анализ графа

Напишем скрипт `scripts/ag_analyze.jl`, который загружает сохранённые данные, строит изображение графа с цветовой индикацией по PageRank и выводит статистику.

```
julia> ag_analyze.jl
using DrWatson
@quickactivate "project"
using Graphs, JLD2, Plots, GraphRecipes
include(scrdir("attack_graph.jl"))

params = Dict{:n => 20, :edge_prob => 0.2, :source => 1, :target => 20}
filename = datadir("attack_graph", savename(params, "jld2"))
@load filename data

g = data[:graph]
metrics = data[:metrics]
pagerank = metrics[:pagerank]

norm_rank = (pagerank .- minimum(pagerank)) ./ (maximum(pagerank) - minimum(pagerank))
colors = [cgrad(:RdYlGn, rev = true)[norm_rank[i]] for i = 1:nv(g)]

graphplot(
    g,
    nodeshape = :circle,
    curves = false,
    linecolor = :black,
    nodecolor = colors,
    nodelabel = 1:nv(g),
    title = "Граф атак (цвет = PageRank)",
    size = (800, 600),
)
```

Рисунок 4.7: Скрипт `ag_analyze.jl`

```
julia> include("scripts/ag_analyze.jl")
График сохранён в C:\Users\Admin\work\study\2026-1\2026-1==s
=== Анализ графа атак ===
Количество узлов: 20
Количество рёбер: 74
Количество путей от 1 к 20: 516
Длины путей: [10, 11, 14, 12, 11, 11, 12, 13, 12, 14, 15, 16, 13, 13, 12, 11, 12, 15, 13, 12, 12, 13, 14, 13, 15, 12, 11, 14, 15, 15, 16, 10, 9, 7, 10, 11, 15, 14, 14, 15, 14, 14, 12, 11, 15, 16, 14, 15, 16, 11, 10, 6, 12, 13, 16, 14, 13, 13, 14, 16, 16, 14, 13, 16, 15, 15, 14, 13, 14, 17, 15, 14, 14, 15, 14, 15, 17, 16, 17, 16, 17, 17, 18, 12, 11, 9, 12, 13, 17, 14, 15, 16, 15, 17, 14, 13, 17, 18, 16, 17, 18, 13, 12, 8, 13, 12, 15, 11, 11, 14, 10, 13, 16, 12, 12, 10, 11, 14, 10, 13, 14, 12, 11, 14, 12, 13, 13, 14, 10, 16, 12, 10, 11, 14, 15]
```

Рисунок 4.8: Результат выполнения `ag_analyze.jl`

```

12, 14, 17, 11, 15, 13, 11, 16, 14, 15, 14, 13, 15, 1
, 12, 14, 16, 12, 10, 15, 17, 13, 11, 14, 13, 16, 15,
Наиболее вероятный путь: [1, 16, 19, 20]

Топ-5 узлов по in-degree:
Узел 6: in-degree = 6
Узел 12: in-degree = 6
Узел 16: in-degree = 6
Узел 1: in-degree = 5
Узел 14: in-degree = 5

Топ-5 узлов по PageRank:
Узел 16: PageRank = 0.0955
Узел 6: PageRank = 0.0832
Узел 1: PageRank = 0.078
Узел 12: PageRank = 0.074
Узел 13: PageRank = 0.0701

```

Рисунок 4.9: Визуализация графа

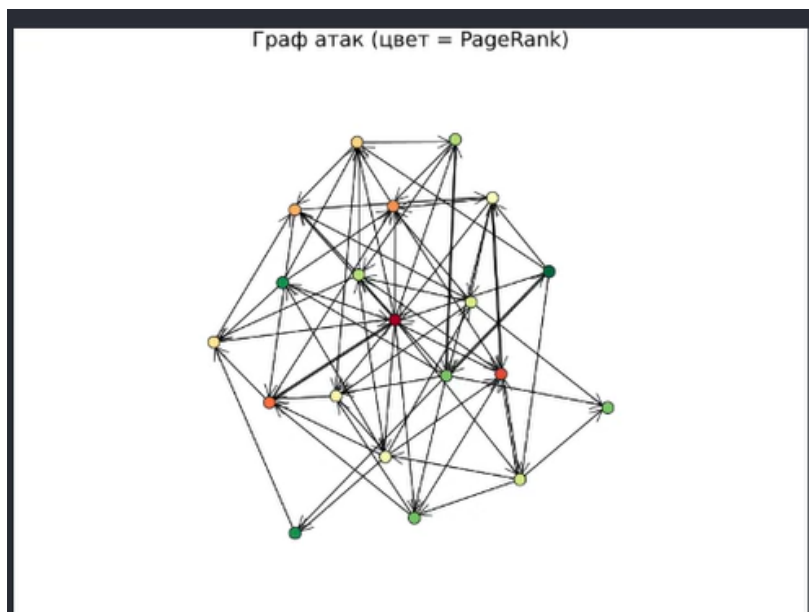


Рисунок 4.10: Визуализация графа

4.5 Исследование масштабируемости

Напишем скрипт `scripts/ag_convergence.jl`, который исследует, как размер сети влияет на время поиска всех путей и количество найденных путей.

```
s > ❖ ag_convergence.jl
julia
using DrWatson
@quickactivate "project"

using Graphs
using Plots
using JLD2
using Random

include(sourcedir("attack_graph.jl"))

sizes = [10, 15, 20, 21, 22, 23, 24, 25]
time_vals = Float64[]
path_counts = Int[]

Random.seed!(123)

for n in sizes
    println("Размер сети: $n")
    g = build_attack_graph(n, 0.2, Dict(), [])
    t = @elapsed paths = find_all_paths(g, 1, n)
    push!(time_vals, t)
    push!(path_counts, length(paths))
    println(" Время: $(round(t, digits=3)) с, путей: $(length(paths))")
end
```

Рисунок 4.11: Скрипт `ag_convergence.jl`

Размер сети: 10
Время: 0.04 с, путей: 1
Размер сети: 15
Время: 0.001 с, путей: 842
Размер сети: 20
Время: 0.001 с, путей: 817
Размер сети: 21
Время: 0.042 с, путей: 58966
Размер сети: 22
Время: 0.365 с, путей: 355669
Размер сети: 23
Время: 1.035 с, путей: 726135
Размер сети: 24
Время: 19.869 с, путей: 10596200
Размер сети: 25
Время: 28.668 с, путей: 19097246

Рисунок 4.12: Рост времени и числа путей

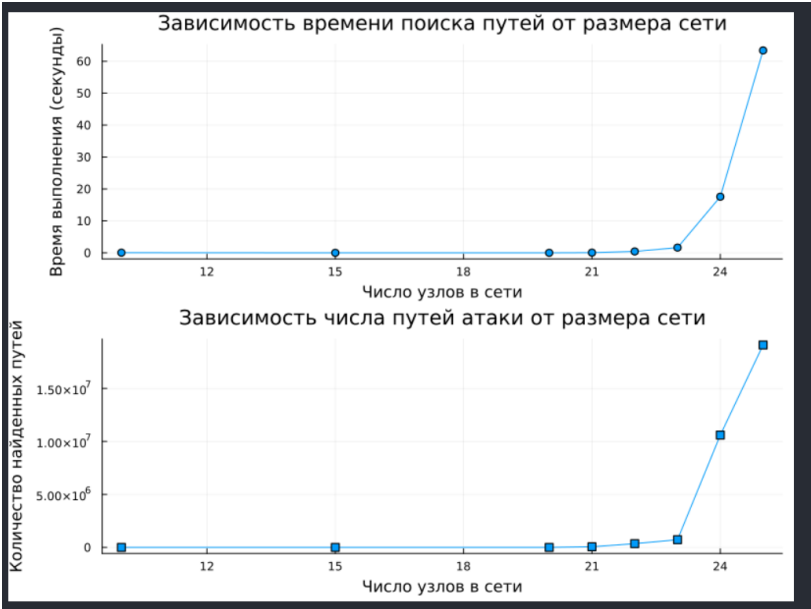


Рисунок 4.13: Рост времени и числа путей

4.6 Параметрическое исследование

Напишем скрипт `scripts/parameter_sweep.jl`, который изучает, как изменение плотности рёбер влияет на количество путей атаки, максимальную входящую степень и среднюю длину пути.

```

s > parameter_sweep.jl
using DrWatson
@quickactivate "project"
using Graphs, Random, CSV, DataFrames, Plots, StatsBase

include(sourcedir("attack_graph.jl"))

edge_probs = 0.1:0.1:0.9
n_nodes = 13
source = 1
target = n_nodes

results = []
Random.seed!(123)

for p in edge_probs
    println("Плотность рёбер: $p")
    g = build_attack_graph(n_nodes, p, Dict{(), ()})
    paths = find_all_paths(g, source, target)
    metrics = compute_centrality_metrics(g)
    max_indeg = maximum(metrics[:in_degree])
    avg_path_len = isempty(paths) ? 0.0 : mean(length.(paths))
    push!(
        results,
        (p = p, paths = length(paths), max_indeg = max_indeg, avg_len = avg_path_len),
    )
end

```

Рисунок 4.14: Скрипт `parameter_sweep.jl`

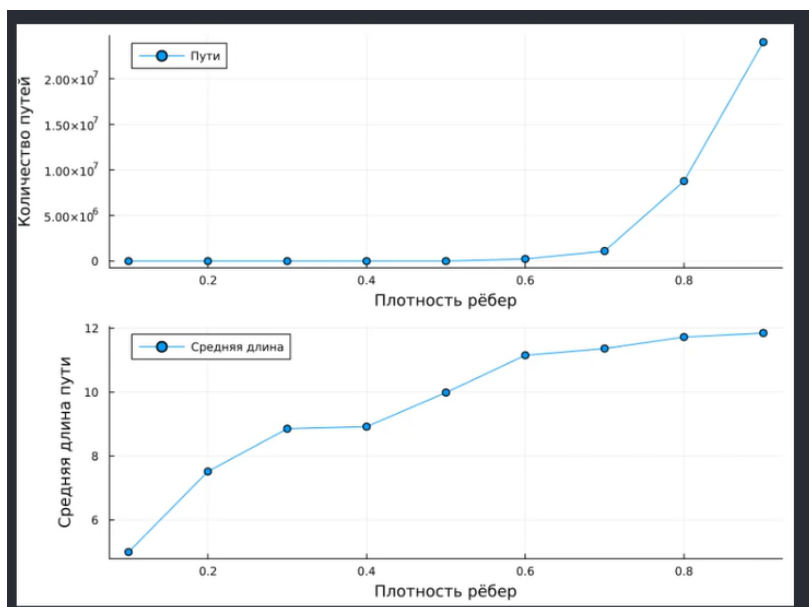


Рисунок 4.15: Зависимость от плотности рёбер

abs > lab02 > project > data > parameter_sweep > results.csv

	A	B	C	D	E
1	edge_...	num_...	max_i...	avg_path_len	
2	0.1	1	3	5.0	
3	0.2	25	7	7.52	
4	0.3	99	7	8.858585858585858	
5	0.4	693	7	8.923520923520924	
6	0.5	4468	8	9.99104744852283	
7	0.6	242776	9	11.1570171680891...	
8	0.7	1109071	11	11.3679160306238...	
9	0.8	87675...	12	11.7277270488637...	
10	0.9	23996...	12	11.8536482872431...	
11					
12					

Рисунок 4.16: Зависимость от плотности рёбер

4.7 Преобразование кода в литературный стиль

Преобразуем код в литературный стиль.

```

lab02 > project > src > literate_attack_graph.jl
#
▷Run | ⏮Above | ⏭Below | ⚙Debug
# ### Функция build_attack_graph
#
# Создаёт ориентированный граф атак с заданными параметрами.
#
# Аргументы:
# - n: количество узлов в сети
# - edge_prob: вероятность существования ребра между двумя узлами
# - vulnerabilities: словарь уязвимостей (задел на будущее)
# - trust_relations: список доверительных отношений в виде кортежей (u, v)

function build_attack_graph(n, edge_prob, vulnerabilities, trust_relations)
    g = SimpleDiGraph{n}

    for i = 1:n
        for j = 1:n
            if i != j && rand() < edge_prob
                add_edge!(g, i, j)
            end
        end
    end

    for (u, v) in trust_relations
        add_edge!(g, u, v)
    end
end

```

Рисунок 4.17: Преобразование кода

Сгенерируем из литературного кода чистый код, Jupyter Notebook и документацию в формате Quarto.


```
scripts > generate_all_reports.jl
24 # Проверяем, какие файлы существуют
25 existing_files = []
26 for file in literate_files
27     if isfile(file)
28         push!(existing_files, file)
29         println("Найден: $file")
30     else
31         println("Не найден: $file")
32     end
33 end
34
35 println("\n" * "="^60)
36 println("Найдено файлов: $(length(existing_files)) из $(length(literate_files))")
37 println("="^60)
38
39 if isempty(existing_files)
40     println("Нет файлов для обработки! Сначала создайте литературные файлы.")
41     exit()
42 end
43
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS Julia REPL (v1.10.0)

=====

Найдено файлов: 5 из 5

=====

Результаты будут сохранены в: C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\lab02\project\literate_attack_graph.jl

Обработка: src/literate_attack_graph.jl

[Info: generating plain script file from `C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\lab02\project\literate\_attack\_graph.jl`

[Info: writing result to `C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\lab02\project\literate\_attack\_graph.jl`

Чистый код: C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\lab02\project\literate_attack_graph.jl

[Info: generating notebook from `C:\Users\Admin\work\study\2026-1\2026-1==study--security-modeling\lab02\project\literate\_attack\_graph.jl`

Рисунок 4.18: Генерация производных форматов

Выполним код из Jupyter Notebook.

Модуль моделирования графов атак

Назначение модуля

Данный модуль содержит все основные функции для работы с графами атак:

- построение графа атак
- поиск всех возможных путей атаки
- вычисление метрик центральности (PageRank, in-degree и др.)
- оценка вероятности успешной атаки

Математическая модель

Граф атак — это ориентированный граф $G = (V, E)$, где:

- $V = \{v_1, v_2, \dots, v_n\}$ — множество узлов сети
- $E \subseteq V \times V$ — множество возможных атак

Ребро $(u, v) \in E$ означает, что атакующий, получив доступ к узлу u , может атаковать узел v .

Рисунок 4.19: Выполнение Jupyter Notebook

```
end

println("=",^60)

[8] ✓ 0.0s

=====
РЕЗУЛЬТАТЫ МОДЕЛИРОВАНИЯ
=====

Количество путей атаки:      5316
Наиболее вероятный путь:    1 → 16 → 19 → 20
Вероятность успеха:         12.5%

Примеры путей атаки (первые 5):
1: 1 → 12 → 6 → 9 → 3 → 2 → 16 → 13 → 5 → 20
2: 1 → 12 → 6 → 9 → 3 → 2 → 16 → 13 → 10 → 5 → 20
3: 1 → 12 → 6 → 9 → 3 → 2 → 16 → 13 → 10 → 15 → 7 → 18 → 19 → 20
4: 1 → 12 → 6 → 9 → 3 → 2 → 16 → 13 → 10 → 15 → 19 → 20
5: 1 → 12 → 6 → 9 → 3 → 2 → 16 → 13 → 10 → 15 → 20
```

Рисунок 4.20: Выполнение Jupyter Notebook

Интегрируем документацию в формате Quarto в отчёт.

```
## Реализация модуля AttackGraph

{{< include ../project/markdown/attack_graph/attack_graph.qmd >}}

## Запуск эксперимента

{{< include ../project/markdown/ag_run_experiment/ag_run_experiment.qmd >}}

## Анализ и визуализация

{{< include ../project/markdown/ag_analyze/ag_analyze.qmd >}}

## Исследование масштабируемости

{{< include ../project/markdown/ag_convergence/ag_convergence.qmd >}}

## Параметрическое исследование

{{< include ../project/markdown/parameter_sweep/parameter_sweep.qmd >}}
```

Рисунок 4.21: Интеграция Quarto

4.8 Изменение кода

Добавим в код в литературном стиле вычисление для набора параметров.

```

labs > lab02 > project > scripts > literate_ag_run_experiment.jl
29
30 # Набор параметров: разные значения плотности рёбер
31 edge_probs = [0.1, 0.12, 0.15, 0.17, 0.2]
32
33 # Фиксированные параметры
34 n_nodes = 20
35 source = 1
36 target = n_nodes
37

```

Рисунок 4.22: Изменение кода

Сгенерируем из литературного кода с параметрами чистый код, Jupyter Notebook и документацию в формате Quarto.

Выполним код из Jupyter Notebook.

```

Плотность рёбер: 0.1
-----
Граф: 20 узлов, 36 рёбер
Найдено путей: 16
Наиболее вероятный путь: 1 → 17 → 13 → 18 → 20
Вероятность успеха: 6.25%

-----
Плотность рёбер: 0.12
-----
Граф: 20 узлов, 58 рёбер
Найдено путей: 565
Наиболее вероятный путь: 1 → 18 → 20
Вероятность успеха: 25.0%

-----
Плотность рёбер: 0.15
-----
Граф: 20 узлов, 61 рёбер
Найдено путей: 3171
Наиболее вероятный путь: 1 → 7 → 20
Вероятность успеха: 25.0%

```

Рисунок 4.23: Выполнение Jupyter Notebook

```

-----
Плотность рёбер: 0.17
-----

Граф: 20 узлов, 61 рёбер
Найдено путей: 1944
Наиболее вероятный путь: 1 → 17 → 11 → 15 → 20
Вероятность успеха: 6.25%

-----
Плотность рёбер: 0.2
-----

Граф: 20 узлов, 76 рёбер
Найдено путей: 7380
Наиболее вероятный путь: 1 → 9 → 20
Вероятность успеха: 25.0%

```

Рисунок 4.24: Выполнение Jupyter Notebook

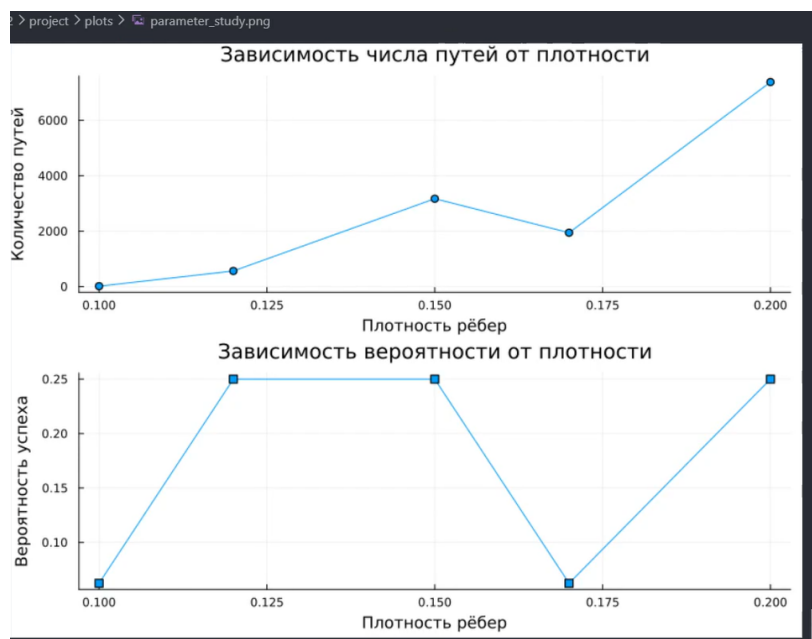


Рисунок 4.25: Выполнение Jupyter Notebook

Интегрируем документацию с параметрами в формате Quarto в отчёт.

4.9 Реализация модуля AttackGraph

```
module AttackGraph
using Graphs using LinearAlgebra using Random
export build_attack_graph, find_all_paths, compute centrality_metrics, assign_edge_weights, most_likely_path, simple_pagerank

function build_attack_graph(n, edge_prob, vulnerabilities, trust_relations) g = SimpleDiGraph(n) for i = 1:n, j = 1:n if i != j && rand() < edge_prob add_edge!(g, i, j) end end for (u, v) in trust_relations add_edge!(g, u, v) end return g end

function find_all_paths(g, source, target) paths = [] function dfs(current, path) if current == target push!(paths, copy(path)) return end for neighbor in outneighbors(g, current) if !(neighbor in path) push!(path, neighbor) dfs(neighbor, path) pop!(path) end end end dfs(source, [source]) return paths end

function compute centrality_metrics(g) indeg = indegree(g) outdeg = outdegree(g) betweenness = betweenness centrality(g) closeness = closeness centrality(g) pagerank = simple_pagerank(g) return Dict{ :in_degree => indeg, :out_degree => outdeg, :betweenness => betweenness, :closeness => closeness, :pagerank => pagerank, } end

function simple_pagerank(g;  $\alpha=0.85$ , max_iter=100, tol=1e-6) n = nv(g) n == 0 && return Float64[] pr = fill(1.0 / n, n) for _ in 1:max_iter pr_new = fill((1 -  $\alpha$ ) / n, n) for i in 1:n outdeg = outdegree(g, i) if outdeg > 0 for j in outneighbors(g, i) pr_new[j] +=  $\alpha$  * pr[i] / outdeg end else for j in 1:n pr_new[j] +=  $\alpha$  * pr[i] / n end end end diff = maximum(abs.(pr_new - pr)) pr = pr_new if diff < tol break end end return pr end

function assign_edge_weights(g, cvss_scores) weights = Dict{Graphs.Edge, Float64}() for e in edges(g) u, v = src(e), dst(e) key = (u, v) weight = get(cvss_scores, key, 0.5) weights[e] = weight end return weights end

function most_likely_path(g, source, target, weights) n = nv(g) distmx = fill(Inf, n, n) for e in edges(g) u, v = src(e), dst(e) w = weights[e] w = max(w, 1e-10) distmx[u, v] = -log(w) end state = dijkstra_shortest_paths(g, source, distmx) if state.dists[target] == Inf return [], 0.0 end path = Int[] current = target while current != source push!(path, current)
```

```

current = state.parents[current] end push!(path, source) reverse!(path) probability = exp(-
state.dists[target]) return path, probability end
end

```

4.10 Запуск эксперимента

```

using DrWatson quickactivate «project» using Graphs, JLD2, Random include(srcdir(«attack_graph.jl»)) using .AttackGraph

params = Dict( :n => 20, :edge_prob => 0.2, :source => 1, :target => 20, :cvss_scores =>
Dict((1,3) => 0.9, (2,5) => 0.7, (3,8) => 0.8, (5,20) => 0.95), :trust_relations => [(4,6), (6,10),
(10,15)], )

filename = datadir(«attack_graph», savename(params, «jld2»)) mkpath(datadir(«at-
tack_graph»))

if isfile(filename) load filename data println(«Данные загружены из $filename»)
else g = build_attack_graph( params[:n], params[:edge_prob], params[:cvss_scores],
params[:trust_relations], ) paths = find_all_paths(g, params[:source], params[:tar-
get]) metrics = compute_centrality_metrics(g) weights = assign_edge_weights(g,
params[:cvss_scores]) likely_path, prob = most_likely_path(g, params[:source],
params[:target], weights) data = Dict( :graph => g, :paths => paths, :metrics => metrics,
:weights => weights, :likely_path => likely_path, :probability => prob, ) save filename
data params println(«Результаты сохранены в $filename») end

println(«Количество путей атаки:», length(data[:paths])) println(«Наиболее
вероятный путь:», data[:likely_path]) println(«Вероятность успеха:», data[:probability])

```

4.11 Анализ и визуализация

```

using DrWatson quickactivate «project» using Graphs, JLD2, Plots, GraphRecipes
include(srcdir(«attack_graph.jl»))

```

```

params = Dict(:n => 20, :edge_prob => 0.2, :source => 1, :target => 20) filename =
datadir(«attack_graph», savename(params, «jld2»)) load filename data
g = data[:graph] metrics = data[:metrics] pagerank = metrics[:pagerank]
norm_rank = (pagerank .- minimum(pagerank)) ./ (maximum(pagerank) - mini-
mum(pagerank)) colors = [cgrad(:RdYlGn, rev = true)[norm_rank[i]] for i = 1:nv(g)]
graphplot( g, nodeshape = :circle, curves = false, linecolor = :black, nodecolor = colors,
nodelabel = 1:nv(g), title = «Граф атак (цвет = PageRank)», size = (800, 600), ) savefig(plots-
dir(«attack_graph.png»))
println(«График сохранён в», plotsdir(«attack_graph.png»))
println(«=== Анализ графа атак ===») println(«Количество узлов:», nv(g)) print-
ln(«Количество рёбер:», ne(g)) println( «Количество путей от», params[:source], ” к
«, params[:target],»:», length(data[:paths]), ) if !isempty(data[:paths]) println(«Длины
путей: «, [length(p) for p in data[:paths]]) println(«Наиболее вероятный путь: “, da-
ta[:likely_path]) end
println(«Топ-5 узлов по in-degree:») top_indeg = sortperm(metrics[:in_degree], rev =
true)[1:5] for i in top_indeg println(” Узел $i: in-degree = $(metrics[:in_degree][i])“) end
println(«Топ-5 узлов по PageRank:») top_pr = sortperm(metrics[:pagerank], rev =
true)[1:5] for i in top_pr println(” Узел $i: PageRank = $(round(metrics[:pagerank][i],
digits=4))“) end

```

4.12 Исследование масштабируемости

```

using DrWatson quickactivate «project»
using Graphs using Plots using JLD2 using Random
include(sourcedir(«attack_graph.jl»)) using .AttackGraph
sizes = [10, 15, 20, 21, 22, 23, 24, 25] time_vals = Float64[] path_counts = Int[]
Random.seed!(123)
for n in sizes println(«Размер сети: $n») g = build_attack_graph(n, 0.2, Dict(), []) t =
elapsed paths = find_all_paths(g, 1, n) push!(time_vals, t) push!(path_counts, length(paths))

```

```
println(" Время: $(round(t, digits=3)) с, путей: $(length(paths))" ) end

p1 = plot( sizes, time_vals, marker = :circle, xlabel = «Число узлов», ylabel = «Время (с)»,
title = «Время поиска всех путей», legend = false, ) p2 = plot( sizes, path_counts, marker
= :circle, xlabel = «число узлов», ylabel = «Количество путей», title = «Количество
путей атаки», legend = false, ) combined = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(combined, plotsdir(«convergence.png»))

mkpath(datadir(«convergence»)) data = Dict(:sizes => sizes, :times => time_vals,
:path_counts => path_counts) save datadir(«convergence», «convergence_data.jld2») data
```

4.13 Параметрическое исследование

using DrWatson **quickactivate** «project» using Graphs, Random, CSV, DataFrames, Plots, StatsBase

```
include(sourcedir(«attack_graph.jl»)) using .AttackGraph

edge_probs = 0.1:0.1:0.9 n_nodes = 13 source = 1 target = n_nodes

results = [] Random.seed!(123)

for p in edge_probs println(«Плотность рёбер: $p») g = build_attack_graph(n_nodes,
p, Dict(), []) paths = find_all_paths(g, source, target) metrics = compute_centrality_met-
rics(g) max_indeg = maximum(metrics[:in_degree]) avg_path_len = isempty(paths) ? 0.0 :
mean(length.(paths)) push!( results, (p = p, paths = length(paths), max_indeg = max_indeg,
avg_len = avg_path_len), ) end

df = DataFrame(results)

mkpath(datadir(«parameter_sweep»)) CSV.write(datadir(«parameter_sweep», «re-
sults.csv»), df)

p1 = plot( edge_probs, [r.paths for r in results], marker = :circle, xlabel = «Плотность
рёбер», ylabel = «Количество путей», label = «Пути», ) p2 = plot( edge_probs, [r.avg_len
for r in results], marker = :circle, xlabel = «Плотность рёбер», ylabel = «Средняя длина
пути», label = «Средняя длина», ) combined = plot(p1, p2, layout = (2, 1), size = (800,
600)) savefig(combined, plotsdir(«parameter_sweep.png»))
```


5 Контрольные вопросы

1. Что такое граф атак и для чего он используется?

Граф атак — это ориентированный граф, вершины которого представляют узлы сети или состояния системы, а рёбра — возможные действия атакующего. Используется для анализа уязвимостей, поиска путей атаки, оценки рисков и выявления критичных узлов.

2. Какие алгоритмы поиска путей применяются в анализе графов атак?

- DFS — для нахождения всех путей.
- BFS — для кратчайшего пути.
- Алгоритм Дейкстры — для поиска наиболее вероятного пути с учётом весов рёбер.

3. Что означают метрики центральности (in-degree, PageRank) в контексте безопасности?

- **In-degree** — количество атак, ведущих к узлу. Высокое значение означает, что узел часто атакуют.
- **PageRank** — важность узла, учитывающая важность ссылающихся на него узлов. Высокий PageRank означает критическое звено.

4. Как можно оценить вероятность успешной атаки с помощью весов рёбер?

Вероятность успеха по пути вычисляется как произведение вероятностей успеха на каждом ребре. Для поиска наиболее вероятного пути используется алгоритм Дейкстры с весом, равным отрицательному логарифму вероятности.

5. Какие ограничения имеет модель графа атак в реальных условиях?

- Статичность (не учитывает изменения сети).
- Не учитывает защитные механизмы (МЭ, IDS).
- Игнорирует временные факторы и человеческий фактор.
- Экспоненциальный рост числа путей для больших сетей.

6. Как можно расширить модель для учёта защитных механизмов (например, межсетевые экраны)?

- Добавить блокирующие правила (удалить рёбра).
- Ввести вероятности обнаружения.
- Добавить узлы-защитники (фаерволы, IDS).
- Использовать взвешенные рёбра с учётом защиты.

6 Выводы

В ходе выполнения лабораторной работы освоены методы построения и анализа графов атак для оценки уязвимостей сетевой инфраструктуры. На примере моделирования атак на корпоративную сеть изучены представление сетевой топологии в виде ориентированного графа, алгоритмы поиска путей атаки, расчёт метрик центральности для определения критических узлов, визуализация графа с цветовой индикацией уровня риска и оценка вероятности успешной атаки с учётом сложности эксплуатации уязвимостей.

Список литературы